Introduction to useful Python libraries

Especially in the context of Apache Airflow

Programme

01 **The modern scientific ecosystem of Python for data manipulation**

02 **Scheduling**

03 **The DB API, database and data platform connectors**

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

01

The modern scientific ecosystem of Python for data manipulation

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

The modern scientific Python ecosystem: knowing where to find new libraries…

PyPI (Python Package Index)	The main repository for Python libraries.	https://pypi.org
Github	Git server. Many Python libraries are hosted on GitHub. It's an excellent platform for discovering open-source projects and seeing contributions in real time.	https://github.com/search
Awesome Python	A huge curated collection of frameworks, libraries, tools, tutorials, data science resources, web development packages, AI/ML ecosystems, and DevOps and automation tools.	https://awesome-python.com/
Forums	The r/Python subreddit regularly features discussions about new Python libraries, with feedback and real-world experience shared by the community. Stack Overflow: answers to development questions often include recommendations for popular or emerging libraries. It is also a good indicator of the size and activity of a library's community.	https://www.reddit.com/r/Python https://stackoverflow.com/
Newsletters	The (many) newsletters share discoveries of new Python libraries, most of them on a weekly basis, along with discussions about trending tools.	

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

The modern scientific Python ecosystem… and how to assess their longevity

- **Popularity/adoption** (number of downloads in the 'Downloads' section on PyPI, number of forks and stars on GitHub, mentions in articles and usage in popular projects/established companies)
- **Update frequency** (date of the last update, commit history, regular version cycles, documented changes)
- **Community support and responsiveness** (issues, maintainer response times)
- **Documentation** (completeness, accuracy, clarity, structure…)
- **Dependencies and compatibility** (minimal and well-managed dependencies, tracking recent Python versions)

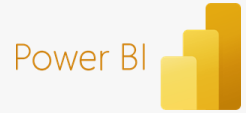
And, even though it's of a different nature, don't forget to inquire about the licence and terms of use!

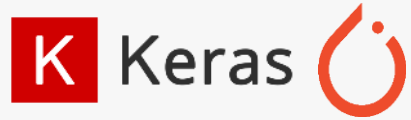
Property of B.S. Total or partial reproduction is prohibited without prior authorization.

The modern scientific Python ecosystem - Beyond the scientific ecosystem, the technical ecosystem for data science

IDE:    jupyterlab 

Data manipulation and processing:     (Vaex) and many others

Data visualisation:  (matplotlib)     

Machine Learning and Deep Learning:     

Database management:    

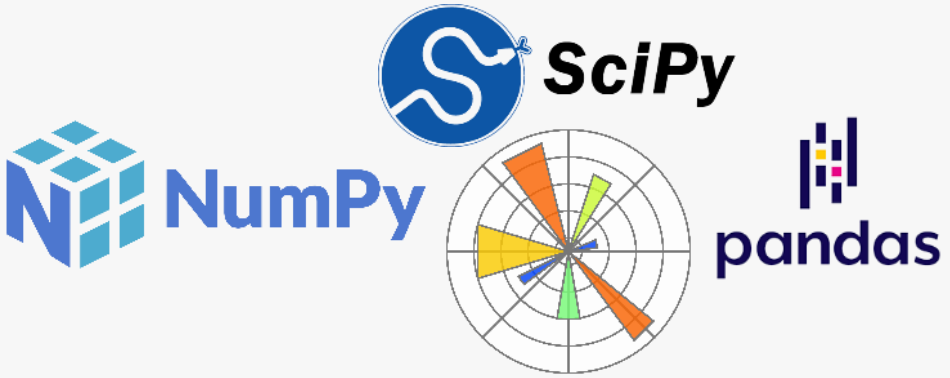
Big Data:    

Version control and collaboration:   

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern Python scientific ecosystem



Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern Python scientific ecosystem - pandas

- Pandas is a data manipulation and analysis library in Python. It provides efficient and easy-to-use data structures, such as DataFrames, which allow for working with tabular data.
- Launched in 2008 by Wes McKinney, Pandas quickly established itself as an essential tool for data science. It is used to import, clean, transform, and analyse heterogeneous datasets.
- Pandas is optimised for handling large datasets thanks to its integration capabilities with NumPy and other Python libraries.

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

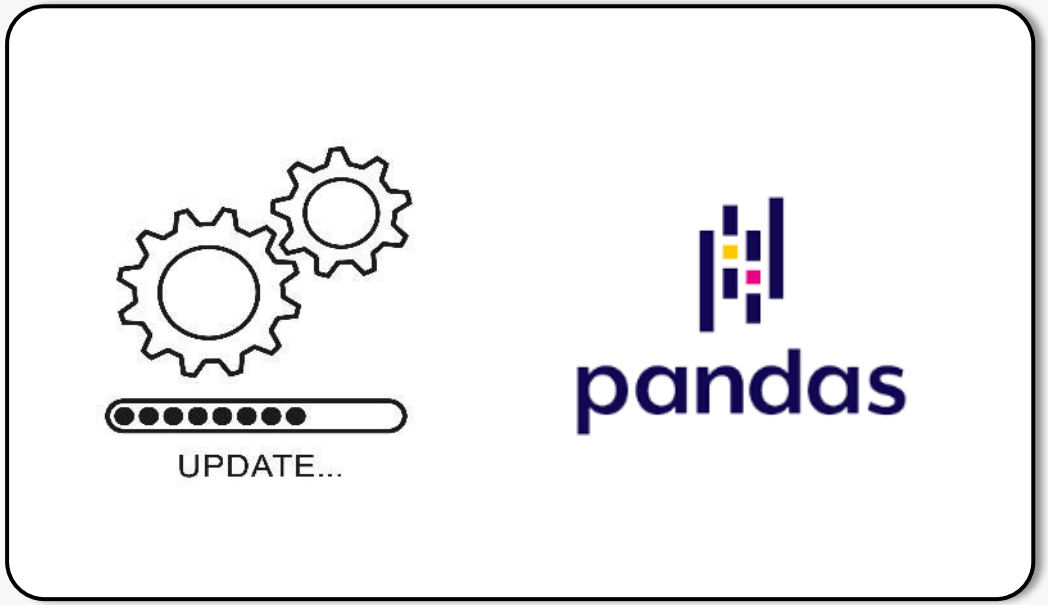
The modern Python scientific ecosystem - pandas

- Pandas DataFrames allow you to manipulate data organised in rows and columns, similar to Excel spreadsheets. Pandas also supports indexing, making data access, sorting, and selection easier.
- The library is ideal for data cleaning tasks, merging datasets, and calculating descriptive statistics. It is often used in conjunction with other tools such as Matplotlib for visualisation or scikit-learn for machine learning.

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern Python scientific ecosystem - pandas




```
pip install --upgrade pandas
```


Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern scientific Python ecosystem - pandas

`df = pandas.DataFrame()` creates a data table.

`df = pandas.read_csv()`, `df = pandas.read_excel()`, `df = pandas.read_sql()`, etc. allow Pandas to interact with external data sources.

`df.head()` displays the first rows, df.tail() displays the last rows.

`df.describe()` provides a statistical summary of the numeric columns.

`df.groupby()` groups the data to apply aggregate operations.

`df.merge()` merges multiple DataFrames.

`df.isna()` identifies missing values, df.fillna() fills them in.

Pandas is compatible with various formats (CSV, Excel, JSON, SQL, HDF5…).

La documentation de pandas est disponible à l'adresse suivante :
https://pandas.pydata.org/docs/

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation - pandas lab #1

Using pandas to manipulate time-series tabular data

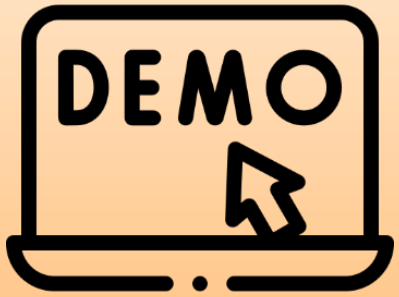
Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation - pandas lab #2

Identifying outliers

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation – distributing pandas-type data processings



Equivalent processing with dask and pyspark

dask PySpark

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation - pandas lab #3

Calculating deltas in a stock portfolio

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern Python scientific ecosystem - NumPy

- NumPy, short for 'Numerical Python', is a fundamental library in Python for numerical computation and the manipulation of multidimensional arrays. It is widely used in various fields, including image processing.

- It was created in 2005 by Travis Oliphant, based on previous work. NumPy is open source and has seen rapid adoption among scientific and engineering communities. It laid the groundwork for many other scientific computing libraries in Python (scipy, matplotlib, scikit-learn…).

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

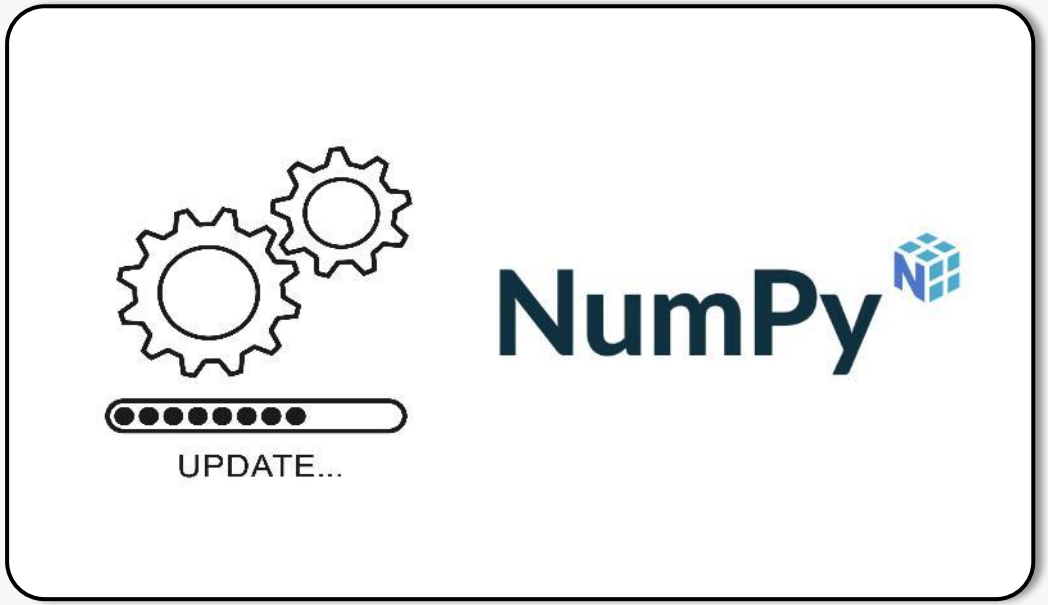
The modern Python scientific ecosystem - NumPy

- NumPy is built around the concept of NumPy arrays (ndarrays), which are homogeneous multidimensional data structures that are efficient for storing and manipulating numerical data.
- NumPy arrays are similar to Python lists, but they are optimised for numerical operations. They can have multiple dimensions and support vectorised operations, making them ideal for numerical computing.
- NumPy provides functions for performing vectorised operations on arrays, allowing efficient and parallel calculations on large amounts of data.
- NumPy is commonly used for processing and analysing data, particularly in the fields of data science, machine learning, and image processing.

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern Python scientific ecosystem - NumPy




```
pip install --upgrade numpy
```


Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern Python scientific ecosystem - NumPy

numpy.array() creates a NumPy array from a sequence of data.

numpy.zeros(), numpy.ones(), numpy.empty() create arrays filled with zeros, ones, or empty values. The size needs to be specified.

numpy.arange(), numpy.linspace() generate sequences of numbers.

numpy.shape(), numpy.reshape(), numpy.dim() allow manipulation of the shape and dimensions of arrays.

numpy.mean(), numpy.max(), numpy.min(), numpy.std() compute statistics on the data.

It is possible to access and manipulate the elements of an array using indices and slicing operations.

La documentation de numpy est disponible à l'adresse suivante :
https://numpy.org/doc/

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation – NumPy lab

Where's Wally?

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern Python scientific ecosystem - SciPy

- SciPy, short for 'Scientific Python', is a library designed for scientific and technical computing. It is built on NumPy and provides a wide range of algorithms for tasks such as integration, optimisation, interpolation, linear algebra, and signal processing.
- Created in 2001 by Travis Oliphant, Eric Jones, and Pearu Peterson, SciPy has played a crucial role in the rise of Python as a leading language for data science and research. It is open source and benefits from an active community of developers and users.
- SciPy is designed to be modular, with several specialised sub-modules such as scipy.integrate (integration), scipy.optimize (optimisation), or scipy.spatial (geometry and distances).

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

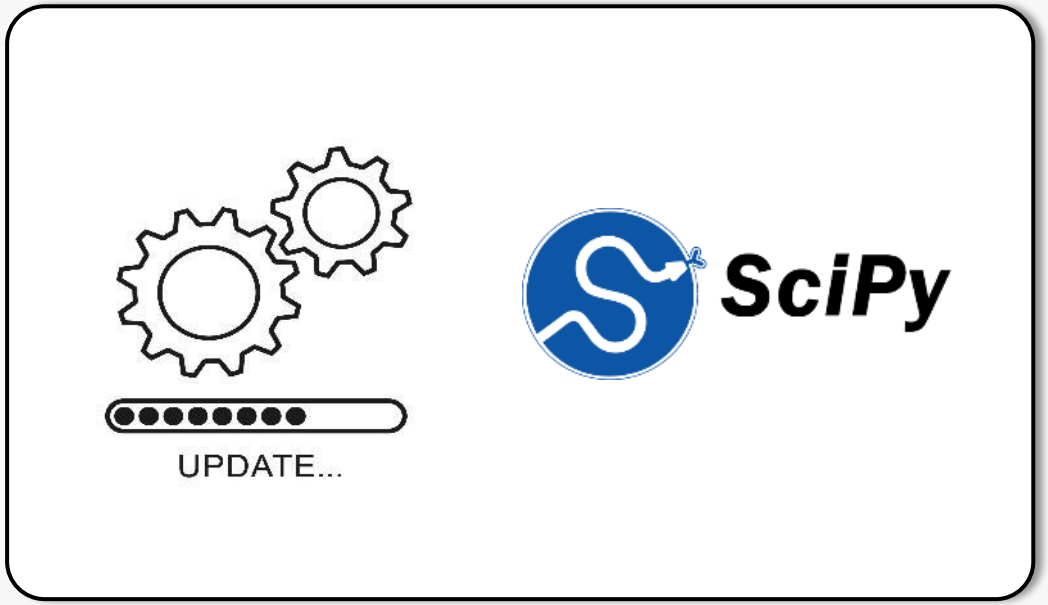
The modern Python scientific ecosystem - SciPy

- SciPy relies on NumPy for data manipulation and focuses on advanced analysis tools. It is commonly used in various fields such as experimental data analysis, physics, bioinformatics, and engineering.
- Its modules allow for solving differential equations, performing advanced matrix decompositions, and analysing statistical distributions. For example, the function scipy.stats provides tools for conducting hypothesis tests or fitting distributions.
- With its optimised performance, SciPy allows for the efficient solving of complex problems.

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern Python scientific ecosystem - SciPy




```
pip install --upgrade scipy
```


Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern scientific Python ecosystem - SciPy

`scipy.optimize.minimize()` solves optimisation problems.

`scipy.integrate.quad()` calculates the integral of a function.

`scipy.interpolate.interp1d()` interpolates data points.

`scipy.linalg.eig()` calculates the eigenvalues of a matrix.

`scipy.stats.norm.pdf()` provides the probability density of a normal distribution.

SciPy also allows manipulation of data structures like sparse matrices via scipy.sparse.

La documentation de scipy est disponible à l'adresse suivante :
https://docs.scipy.org/doc/scipy/

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern scientific Python ecosystem - Matplotlib

- Matplotlib is a flexible data visualisation library for Python. It is used to create a wide variety of graphs, plots, and visualisations, whether for data analysis, presentations, or scientific reports. It is also capable of displaying and processing basic images, making it a versatile tool for visualisation and image processing.

- It was created by John D. Hunter in 2003. He developed this library to help scientists and engineers visualise their data effectively and aesthetically.

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

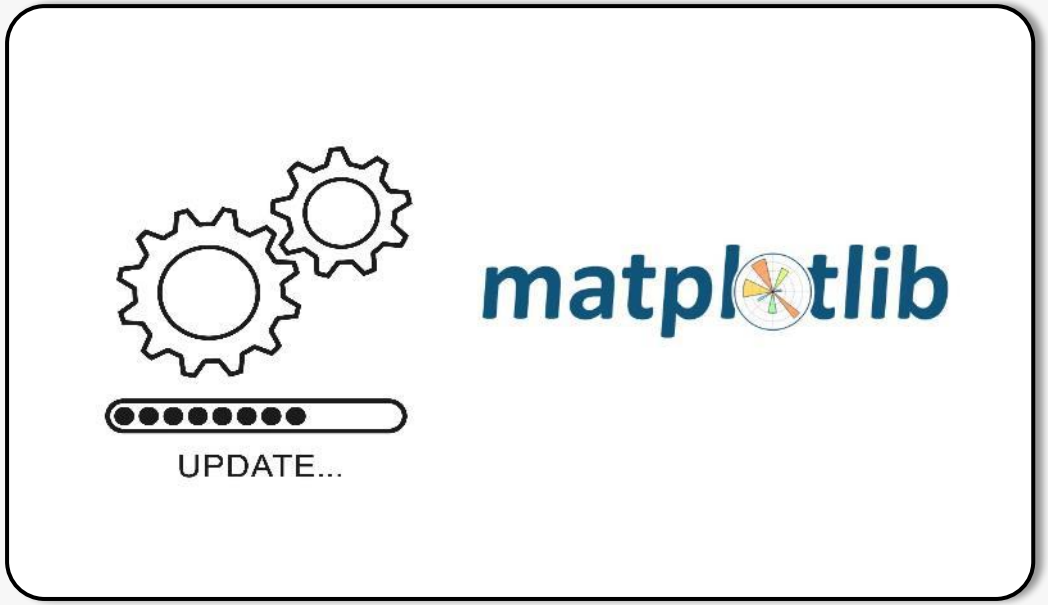
The modern Python scientific ecosystem - Matplotlib

- Matplotlib works by providing an object-oriented interface for creating graphs and plots in Python. The pyplot submodule provides a simplified interface for quickly building visualisations.
- Matplotlib allows users to create figures and axes that can be extensively customised. Users can add elements such as lines, points, legends, titles, etc., to graphs in a 'programmatic' manner.
- Matplotlib supports a wide variety of chart types, including scatter plots, histograms, bar charts, box plots, line graphs, scattergraphs, pie charts, and more.
- Matplotlib allows for advanced customisation of graphs, with the ability to control every visual aspect, including colours, line styles, fonts, labels, axes, and scales.
- It is often used with Pandas for plotting tabular data or with NumPy to visualise raw numerical data.

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern Python scientific ecosystem - Matplotlib




```
pip install --upgrade matplotlib
```


Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation

The modern scientific Python ecosystem - Matplotlib

`plt.figure()`, `plt.subplot()` allow you to create figures and subplots.

`plt.plot()`, `plt.scatter()`, `plt.bar()` create different types of charts.

`plt.xlabel()`, `plt.ylabel()`, `plt.title()`, `plt.legend()` add legend/title elements to the visualisation

`plt.savefig()` saves the chart to an image file.

Official documentation :
https://matplotlib.org/stable/index.html

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data visualisation - Lab

Vizualising the location of French and American airports on a map

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Data manipulation – another lab

Apple stock price data

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

The modern Python scientific ecosystem

The modern Python scientific ecosystem - scikit-learn

scikit-learn is an (excellent) Python library for Machine Learning, providing a set of standardised tools for supervised and unsupervised learning. It is commonly used as a catalogue of implemented algorithms and, more generally, for building models, transforming data, and evaluating the performance of Machine Learning algorithms.

Data Preprocessing	Machine Learning Algorithms	Model Evaluation	Model optimisation
scikit-learn includes methods for normalising, transforming, and manipulating data to prepare it for the model	**scikit-learn offers a (wide) range of models for tasks such as classification, regression, clustering, dimensionality reduction, and model selection**	**scikit-learn provides functions to evaluate model performance using standard metrics and cross-validation techniques**	**scikit-learn allows for the search of the best hyperparameters for a model using techniques such as GridSearchCV and RandomizedSearchCV**

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

02

Scheduling

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Scheduling

Date and time handling

datetime

Part of Python's standard library and is used to:

- Get the current date/time
- Parse and format timestamps
- Perform date arithmetic
- Work with time zones (via zoneinfo in modern Python)

```
from datetime import datetime, timedelta

now = datetime.now()
next_run = now + timedelta(hours=1)
```


Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Scheduling

Date and time handling

pendulum

Third-party library that provides a more user-friendly API than datetime.

- Better timezone support.
- Easier date arithmetic.
- More intuitive parsing and formatting.

```
import pendulum
now = pendulum.now("Europe/Paris")
next_run = now.add(hours=1)
```


Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Scheduling

Date and time handling

asyncio

Async (asyncio) is not a scheduling library either, but it can be used to schedule and run tasks within an asynchronous application.

When building an async application (e.g., with FastAPI), you may want tasks to run periodically without blocking the application.

```
import asyncio

async def periodic_task():
    while True:
        print("Running task...")
        await asyncio.sleep(60) # wait 60 seconds

asyncio.run(periodic_task())
```


Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Scheduling

Scheduling libraries

schedule

Simple, lightweight, and easy to use.

```
import schedule
import time

def job():
    print("Running task")

schedule.every(10).minutes.do(job)

while True:
    schedule.run_pending()
    time.sleep(1)
```


Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Scheduling

Scheduling libraries

APScheduler

Scheduler supporting:

- Cron-style schedules
- Fixed intervals
- Specific dates/times
- Persistence via databases

```
from apscheduler.schedulers.blocking import BlockingScheduler

scheduler = BlockingScheduler()

@scheduler.scheduled_job('cron', hour=9)
def morning_task():
    print("Good morning!")

scheduler.start()
```


Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Scheduling

Scheduling libraries

Celery + Celery Beat

Distributed task queue that supports scheduled jobs through Celery Beat.

- Background workers
- Retry mechanisms
- Distributed execution
- Scales across multiple servers

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Scheduling

Scheduling libraries

Cron (via Python)

On Linux, many teams use system cron jobs instead of a Python scheduling library (server-side schedule for scripts).

```
0 9 * * * /usr/bin/python3 /path/to/script.py
```


Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Scheduling

Scheduling libraries

RQ Scheduler

Built on top of Redis Queue (RQ), best for Redis-based applications and simpler alternative to Celery.

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Scheduling

Scheduling libraries

schedule

Simple, lightweight, and easy to use.

```
import schedule
import time

def job():
    print("Running task")

schedule.every(10).minutes.do(job)

while True:
    schedule.run_pending()
    time.sleep(1)
```


Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Scheduling

Async + APScheduler

APScheduler supports async applications and can schedule async functions directly.

```
from apscheduler.schedulers.asyncio import AsyncIOScheduler

scheduler = AsyncIOScheduler()

async def task():
    print("Hello")

scheduler.add_job(task, "interval", minutes=5)
scheduler.start()
```

This is a common setup in FastAPI applications for instance.

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Scheduling

Date and time handling

In the context of Apache Airflow, some of these libraries are very relevant, while others overlap with functionality that Airflow already provides.

Relevance	Libraries
Highly relevant	?
Somewhat relevant	?
Usually unnecessary	?

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

Scheduling

Date and time handling

In the context of Apache Airflow, some of these libraries are very relevant, while others overlap with functionality that Airflow already provides.

Relevance	Libraries
Highly relevant	datetime, pendulum
Somewhat relevant	asyncio
Usually unnecessary	schedule, APScheduler

Celery is an interesting special case. You typically don't use Celery *inside DAG code*, but Airflow can use a Celery Executor to distribute tasks across workers. If you are administering Airflow infrastructure, Celery knowledge is valuable. If you are just writing DAGs, much less so.

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

03

The DB API, database and data platform connectors

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

The DB API

Access to relational databases, the operation of the DB API

DB API 2.0 (PEP 249): a Python standard that defines a common interface for interacting with all relational databases (SQLite, PostgreSQL, MySQL, Oracle...).

Objective: to write Python code that is as database-independent as possible.

Advantages:

- **Uniformity: same interface for SQLite, PostgreSQL, MySQL...**
- **Security: placeholders to prevent SQL injections**
- **Transaction control: commit() and rollback()**
- **Ease of use with cursors and fetch methods**

SGBD	Module Python
SQLite	sqlite3
PostgreSQL	psycopg2
MySQL	mysql-connector-python
Oracle	cx_Oracle

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

The DB API

Access to relational databases, the functioning of the DB API

General steps:

1. Import the corresponding Python module for the database (SQLite, PostgreSQL, MySQL, etc.).
2. Create a connection to the database.
3. Create a cursor from the connection.
4. Execute SQL commands via the cursor (SELECT, INSERT, UPDATE, DELETE, etc.).
5. Retrieve the results of queries if necessary (fetchone(), fetchall()).
6. Commit changes with commit() for commands that modify the database.
7. Handle errors and optionally perform a rollback() in case of an exception.
8. Close the cursor and the connection to the database.

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

The DB API

Access to relational databases, the functioning of the DB API

Concept	Description
Connection	Object representing the database connection. It allows the creation of cursors and the management of transactions.
Cursor	An object used to execute SQL queries and retrieve results.
Placeholders	They help prevent SQL injection. Syntax: ? for SQLite, %s for PostgreSQL/MySQL.
commit()	Validates and commits changes made by INSERT, UPDATE, and DELETE operations.
rollback()	Reverts any uncommitted changes if an error occurs.
fetchone() / fetchall()	Methods for retrieving the results of a SELECT query.

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

The DB API – Working with SQL databases

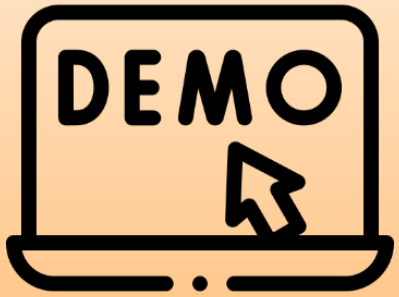
Lab

sqlite3 for SQLite

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

The DB API

Demo



Psycopg2 for POSTGRESQL

Property of B.S. Total or partial reproduction is prohibited without prior authorization.

End of this section

Thank you!

Property of B.S. Total or partial reproduction is prohibited without prior authorization.